

Introduction to Programming

Problem Sheet 1

No imports should be required

Section 1

1. Identity function on Integers.

```
id' :: Integer -> Integer
id' = undefined
```

2. Project first element of a pair of Integers.

```
fst' :: (Integer, Integer) -> Integer
fst' = undefined
```

3. Project second element of a pair of Integers.

```
snd' :: (Integer, Integer) -> Integer
snd' = undefined
```

4. Integer 1-digit adder. Add two single-digit integers (0-9). (Assume both inputs are in the range (0-9).) The first component should have the unit digit of the answer, and the second component should be the carry-over amount

```
addDigit :: Integer -> Integer -> ( Integer , Integer )
addDigit = undefined
```

Section 2

6. Maximum of two Integers.

```
max' :: Integer -> Integer -> Integer
max' = undefined
```

7. Minimum of two Integers.

```
min' :: Integer -> Integer -> Integer
min' = undefined
```

8. Vector addition: add two 2D vectors component-wise. Vector represented as (x, y).

```
vecAdd :: (Integer, Integer) -> (Integer, Integer) -> (Integer, Integer)
vecAdd = undefined
```

9. Scalar multiplication: multiply a 2D vector by a scalar.

```
scalarMul :: Integer -> (Integer, Integer) -> (Integer, Integer)
scalarMul = undefined
```

Section 3

10. Matrix addition (2×2 matrices represented as two rows).

```
matAdd :: ((Integer, Integer), (Integer, Integer))
        -> ((Integer, Integer), (Integer, Integer))
        -> ((Integer, Integer), (Integer, Integer))
matAdd = undefined
```

11. Matrix transpose (swap rows and columns).

```
transpose :: ((Integer, Integer), (Integer, Integer))
           -> ((Integer, Integer), (Integer, Integer))
transpose = undefined
```

Section 4

12. Matrix multiplication (2×2).

```
matMul :: ((Integer, Integer), (Integer, Integer))
        -> ((Integer, Integer), (Integer, Integer))
        -> ((Integer, Integer), (Integer, Integer))
matMul = undefined
```

Section 5

13. Pair combinator (was called "fork"). Given two functions $f :: \text{Integer} \rightarrow \text{Integer}$ and $g :: \text{Integer} \rightarrow \text{Integer}$, produce a function that takes an $\text{Integer } x$ and returns $(f\ x, g\ x)$.

```
pair :: (Integer -> Integer) -> (Integer -> Integer)
      -> ( Integer -> (Integer, Integer) )
pair = undefined
```

Section 6

5. Recall the bijection between $A^{(B \times C)}$ and $(A^B)^C$ Given the following function,

```
myFunc :: (Integer, Integer) -> Integer --  $A^{(B \times C)}$  where  $A = B = C = \text{Integer}$ 
myFunc (x,y) = x*x*y + y*(x*x+1) - 120*(y - x*y)
```

there must be a corresponding function

```
corrFunc :: Integer -> ( Integer -> Integer ) --  $(A^B)^C$  where  $A = B = C = \text{Integer}$ 
corrFunc = undefined
```

14. Bijection between Integer and $(\text{Integer}, \text{Integer})$ Define a bijection (one-to-one and onto) between integers and pairs of integers. That is, give functions $\text{toPair} :: \text{Integer} \rightarrow (\text{Integer}, \text{Integer})$ and $\text{fromPair} :: (\text{Integer}, \text{Integer}) \rightarrow \text{Integer}$ such that they are inverses.

```
toPair :: Integer -> (Integer, Integer)
toPair = undefined

fromPair :: (Integer, Integer) -> Integer
fromPair = undefined
```